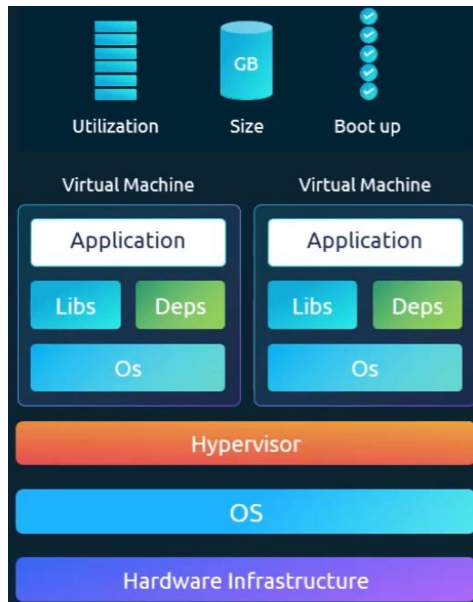
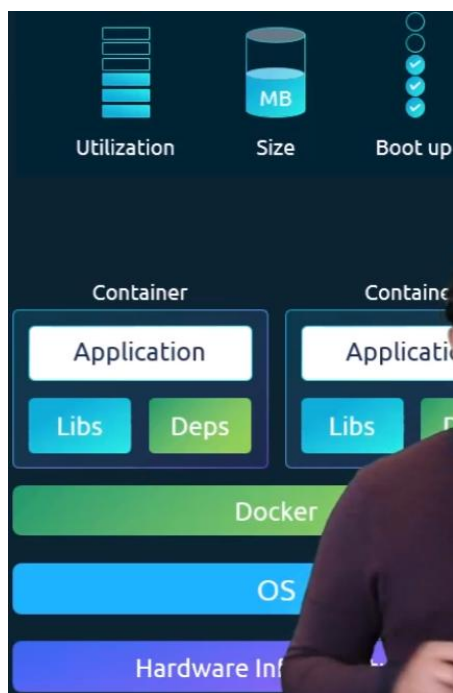


- Docker containers share the same Kernel of the host machine.
 - ⌘ If the system has linux kernel, then you can't run windows container in that system.
 - ⌘ The OS of the container should be compatible with the kernel of the host machine.



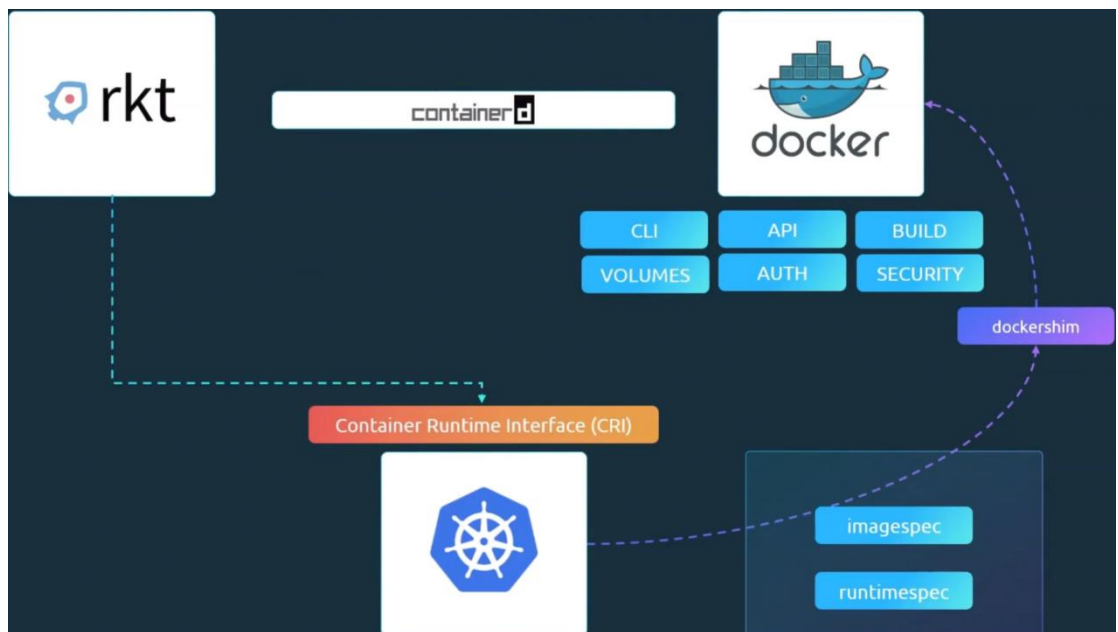
(virtual machine)



(Docker)

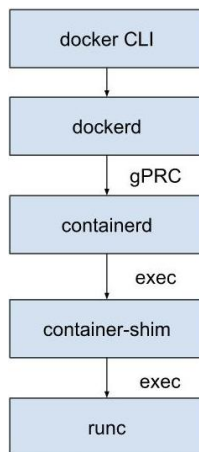
- When you install Kubernetes in a system, then the following things are installed:
 - ♣ API Server
 - ☞ It acts as the frontend of kubernetes cluster.
 - ♣ etcd
 - ☞ Distributed key-value store that keeps the cluster information(i.e. all details about worker nodes and master nodes) of kubernetes to manage the cluster.
 - ♣ Kubelet
 - ☞ Make sure containers are running inside the nodes.
 - ♣ Container Runtime
 - ☞ Underlying software used to run software.
 - ☞ Docker has “containerd”.
 - ♣ Controller
 - ☞ Brain behind the orchestration.
 - ☞ It takes decision to bring up new containers if nodes or pods goes down.
 - ♣ Scheduler
 - ☞ Implementing locks within the cluster to ensure there are no conflicts.
 - ☞ Assign the nodes to the pods.
- How does nodes are divided into master and slave?
 - ♣ Each worker node contains *Container Runtime* like containerd.
Other container runtime are also available like *rkt*, *CRI-O*.
 - ♣ Master node contains **kube-apiserver** and Worker node contains **kubelet** ,
this kube-apiserver makes the node master.
 - ♣ **Kubelet** agent of the worker node connects with the master node to do the things.
- Basic commands:
 - ♣ **kubectrl run hello-minikube**
kubectrl run command is used to deploy an application to the cluster.
 - ♣ **kubectrl cluster-info**
it gives the information about the **cluster**.
 - ♣ **kubectrl get nodes**
it list all the nodes of the cluster.
- Docker vs containerd
 - ♣ Kubernetes introduced an interface called CRI (container runtime interface)
 - ♣ CRI allowed every vendor to work as a container runtime for kubernetes as long as they follow the OCI (open container initiative) standard.

- ⌘ This CRI has 2 specs:
 - ⌘ imagespec
 - it is the specification about how an image should be built.
 - ⌘ runtimespec
 - it is the specification about how any container runtime should be developed.
- ⌘ Docker was there way before CRI was introduced. So, Kubernetes introduced ***dockershim*** to use Docker which was a temporary solution.
- ⌘ Now, *Docker Runtime* (containerd) contains inbuilt CRI.



- ⌘ Docker has multiple things that work like docker cli, api, volumes etc etc.
- ⌘ **containerd** is CRI compatible, can be used as a container runtime separately with Kubernetes.
- ⌘

➤ **NOTE about Docker architecture layers** -----



➤ When you run **docker run nginx**, internally the below happens

- ⌘ **Docker CLI** sends request to **dockerd**.
- ⌘ **dockerd** (*docker daemon*) does image pull, container setup, volume setup, container config.

dockerd is the brain of docker.

Then, it calls **containerd**.

- ⌘ **containerd** calls **runc**.

It actually creates a container (namespace, cgroups)

➤ **docker = dockerd (manager) + containerd (executor) + runc (low-level runner)**

➤ The parts of docker:

- ⌘ Docker CLI
 - ⌘ the **docker** command you type.
 - ⌘ It talks to the docker daemon (**dockerd**) via REST API.
- ⌘ Docker Daemon (**dockerd**)
 - ⌘ Its brain of Docker.
 - ⌘ It handles containers, images, networks, volumes.
 - ⌘ Exposes API for CLI/Tools.
- ⌘ Docker API
 - ⌘ REST API used by CLI, or external tools.
 - ⌘ Ex: Docker Desktop, CI tools
- ⌘ Docker Build System
 - ⌘ Builds images from *Dockerfile*.
 - ⌘ Internally uses **Buildkit** (modern Docker).
- ⌘ Docker objects (managed by Docker)
 - ⌘ Containers, volumes, networks, images.

➤ The parts outside of Docker.

⌘ **Container Runtime** (high level)

- ⌘ containerd, it manages pulling images, container lifecycle.
- ⌘ Originally part of docker, now a separate CNCF project.

⌘ **Low Level Runtime**

- ⌘ **runc**
- ⌘ Actually creates containers using namespace, cgroups.

➤ How images are built?

- ⌘ Docker is something that calls **containerd** which further calls **runc** to create an image.

- ⌘ Any vendor can use **containerd** and create image.

- ⌘ It doesn't mean that docker is just a tool like others. It's a complete platform which contains

- ⌘ CLI (docker)
- ⌘ Daemon (dockerd)
- ⌘ Image build system (buildkit)
- ⌘ Networking, Volumes, Image management

- ⌘ It internally uses **containerd** and **runc**.

- ⌘ **containerd**: pull, start, stop the container

runc: actually runs the container (namespaces, cgroups)

- ⌘ Before

kubernetes -- dockershim -- Docker -- containerd -- runc

- ⌘ Now

kubernetes -- containerd -- runc

- ⌘ Kubernetes removed docker completely from its pipeline.

- ⌘ You just build the image and push in the registry, rest of the things will be done by Kubernetes.

Because anyways it uses **containerd** which can pull and delegate the call to **runc** to run the container.

Docker now adapted to CRI standard, so it builds CRI standard images that can be directly used by Kubernetes.

- ⌘ Docker is having Docker Networks, Volumes, Lifecycle, **How will they all be managed without Docker in Kubernetes?**

- ⌘ For networking, Kubernetes has **CNI** which is very powerful.

- ✧ For Storage, PersistentVolumes, PersistentVolumeClaims, StorageClasses are there.
- ✧ For lifecycle, pods and replica management is there.
- Kubernetes has **crictl** to talk to containerd in runtime for debugging purpose.
 - ✧ Docker doesn't use *crictl* to talk to containerd (don't get confused on this)
 - ✧ Both **dockerd** and **crictl** talks to *containerd* but in different ways.
 - ✧ *dockerd* talks to *containerd* via its own API (gRPC)
 - ✧ *crictl* talks to *containerd* via CRI plugin (*CRI API*)

```
docker CLI = developer tool (build + run)
crictl      = debugging tool (inspect runtime)
```

docker cli	crictl	Description	Unsupported Features
attach	attach	Attach to a running container	--detach-keys , --sig-proxy
exec	exec	Run a command in a running container	--privileged , --user , --detach-keys
images	images	List images	
info	info	Display system-wide information	
inspect	inspect , inspecti	Return low-level information on a container, image or task	
logs	logs	Fetch the logs of a container	--details
ps	ps	List containers	
stats	stats	Display a live stream of container(s) resource usage statistics	Column: NET/BLOCK I/O, PIDs
version	version	Show the runtime (Docker, ContainerD, or others) version information	

- ✧ **crictl** can connect to any *CRI* configured runtime, not only *containerd*.
- F

➤ Minikube

- ⌘ It provides a single node environment where all the processes of both control plane and data plane are present in a single node.



- ⌘ it provides an iso image directly that can be run using any virtual box like VM ware, virtualbox ..etc.
- ⌘ **minikube start --driver=virtualbox** (use --driver to run minikube inside virtual machine)
if you are using WSL then virtualbox vms will not work because it cannot directly contact the hyperv, so, instead give the --driver=docker.
If you remove WSL then Docker will not work.

➤ Pods

- ⌘ The Nodes doesn't run the application directly. Rather the container is wrapped with an object called as **pod**.
 - ⌘ It is the smallest object that can be created in Kubernetes.
 - ⌘ Do not add multiple containers inside a single pod if you want to scale, rather create different pods for that.
And for CPU or other usage, then run the pods in multiple nodes.
 - ⌘ Its not a restriction that we can create only one container inside a pod, we can create many.
But, those containers **shouldn't be of same kind**.
There are use-cases that create helper containers along with the application container inside the same pods which lives and dies along with the main application container.
 - ⌘ The application and helper containers can communicate with eachother.
- Why Helper containers?
- ⌘ Lets see only docker containers for now.

- ⌘ You have some application containers. But as your app grows, you need to process some data.
So, you created some other containers to process the respective containers.
- ⌘ Now, you need to keep shared volumes for each application and helper container pairs.

```
docker run python-app
docker run python-app
docker run python-app
docker run python-app
docker run helper --link app1
docker run helper --link app2
docker run helper --link app3
docker run helper --link app4
```

- ⌘ (-link is the legacy method to link one container to another in docker)

Now, you need to keep track of the containers like

App	Helper	Volume
Python1	App1	Vol1
Python2	App2	Vol2

- ⌘ With **Pods**, Kubernetes does all these by default.
All the containers inside a Pod can access each other, can access the same volume, and created and destroyed together.
 - ⌘ Multiple containers are very rare use-case.
- To create a **Pod**

```
C:\Users\alokr>kubectl run nginx --image=nginx
pod/nginx created

C:\Users\alokr>kubectl get pods
NAME      READY   STATUS             RESTARTS   AGE
nginx     0/1     ContainerCreating   0           6s

C:\Users\alokr>kubectl get pods
NAME      READY   STATUS             RESTARTS   AGE
nginx     0/1     ContainerCreating   0           13s

C:\Users\alokr>kubectl get pods
NAME      READY   STATUS    RESTARTS   AGE
nginx     1/1     Running   0           27s
```


• **kubect**l **run** <name> --image=<image name>

it'll pull the image from docker registry and create a pod and run the container inside that.

```
C:\Users\alokr>kubectl describe pod nginx
Name:          nginx
Namespace:     default
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Thu, 23 Apr 2026 03:12:38 +0530
Labels:        run=nginx
Annotations:    <none>
```

kubectl **describe pod** <name>

it displays a lot of details about the pod.

Also, one more command is there **kubect**l **get pods -o wide**

```
C:\Users\alokr>kubectl get pods -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP           NODE      NOMINATED NODE   READINESS GATES
nginx     1/1     Running   0          7m6s  10.244.0.3   minikube  <none>           <none>
```

Pods with YAML

```
apiVersion: v1
kind: Pod
metadata:
  name: myapp-pod
  labels:
    app: myapp
    type: front-end
spec:
  containers:
  - name: nginx-container
    image: nginx
```



^ apiVersion

- It's the Kubernetes API version, not your app's version.

Because the app version will be there in the docker image like *my-app:v1* .

- Kubernetes API** and **kube-apiserver** both are different.

kube-apiserver exposes the *Kubernetes API* .

analogy: [kube-apiserver: nodejs application, kubernetes API: the exposed REST end-points](#).

```
apiVersion: v1
kind: Pod
```



internally it becomes: **/api/v1/pods**

API Version	Used For	Resources
v1	Core/basic resources	Pod, Service, ConfigMap, Secret
apps/v1	Workload controllers	Deployment, ReplicaSet, StatefulSet, DaemonSet
batch/v1	One-time / scheduled tasks	Job, CronJob
networking.k8s.io/v1	Networking	Ingress, NetworkPolicy
rbac.authorization.k8s.io/v1	Permissions & access control	Role, RoleBinding, ClusterRole
autoscaling/v2	Auto scaling	HorizontalPodAutoscaler
storage.k8s.io/v1	Storage management	StorageClass, VolumeAttachment



^ kind

- Its like what kind of object do you want to create.

- Simple analogy:

apiVersion: which rule you want to apply?

kind: applying that rule, what do you want to do?

kind	Meaning
Pod	Run a container
Deployment	Manage multiple Pods
Service	Expose Pods
ConfigMap	Store config
Secret	Store sensitive data
Job	Run task once

☞

metadata

- There are some specific keys like *name, labels, namespace, annotations ..etc* which kubernetes expects to be inside metadata, other than those accepted keys, you shouldn't give anything.
- Under *labels*, you can give your own key-value pairs you want.

specs

- spec = what you want Kubernetes to actually do (the desired state)

```
spec:
```

- `replicas: 3`, it says -- I want 3 pods running.

- Specs defines: which container? which image? how it should run?

- In simple terms: *specs is the desired state. Controllers (inside controller-manager) work to make actual state match it.*

- You can see, *containers* is a list.

If you give multiple container name and image, then it'll run that many containers inside a single pod.

```
containers:
  - name: app
    image: nginx
  - name: sidecar
    image: busybox
```

- Deployment controller → ensures correct number of Pods
- Node controller → handles node health
- ReplicaSet controller → maintains replicas

- Inside *containers*, you'll give list of dictionaries each containing *name* and *image*.

Image: the actual container image that is pulled from registry.

Name: just a name of that container inside Kubernetes.

- ⌘ At the end, run the command

```
kubectl create -f <yml file name>
```

```
kubectl apply -f <yaml file name>
```

create doesn't update, just create the pods if doesn't exist and don't touch it later. If exist then error.

apply update the existing config (if present) or create the new configs (if not present) according to yaml. If you change yaml further, then you can again call *apply* but not *create*.

Always use **apply** instead of *create*.

- ⌘ **kubectl describe pod <pod name>**

it displays all the details about the pod.

- ⌘ **kubectl run nginx --image nginx**

you can create pods using **run** command as well, if you want to debug something or create a pod quickly then it can be used.

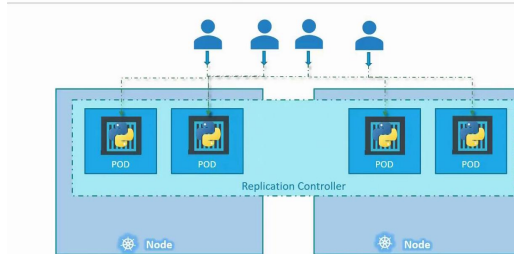
apply or **create** gives control because you can configure the things in yaml or any other file.

- F

Replication Controllers and ReplicaSets

- Controller Manager (CM) consists of different types of controllers, one of them is **Replication Controller**.
- **Replication Controller**
 - ♣ It keeps the desired number of pods running.
 - ♣ If one pod goes down, it build another to keep the desired state.
 - ♣ Does that mean, if you want only one then you can't use Replication Controller?
NO, if that single pod goes down, then Replication Controller build another pod in same or different node to keep the application running.
 - ♣ It also helps in scaling by building pods through out the cluster.

Load Balancing & Scaling



```
# rc-definition.yml
apiVersion: v1
kind: ReplicationController
metadata:
  name: myapp-rc
  labels:
    app: myapp
    type: front-end
spec:
  replicas: 3
  template:
    metadata:
      name: myapp-pod
      labels:
        app: myapp
        type: front-end
    spec:
      containers:
        - name: nginx-container
          image: nginx
```

- ♣ It is confusing that, when the kind was 'Pod', we were writing *containers* inside the *spec* and that's it.

⌘ But here, there is a *spec* inside *spec*.

⌘ Here, we are not using **Pods** directly, instead we are telling Kubernetes to create the pods.

⌘ First level is for *Replication Controller*, which will manage the pods.

So, inside that, we are giving the template of the *Pod*.

⌘ That **template** key inside the Spec will be exact same as the yaml configuration of kind *Pod*.

The difference is, inside that template you don't write *apiVersion*, *kind*.

```
ReplicationController
├── spec (RC config)
│   ├── replicas: 3
│   └── template
│       ├── spec (Pod config)
│       └── containers
```

⌘ Run the same command i.e.

kubectl apply/create -f <yaml file name>

to run the *Replication Controller*.

⌘ You can see in the yaml, the replication controller's name is *myapp-rc* (against metadata.name), so **all the pods will have name with prefix *myapp-rc***.

➤ ReplicaSets

⌘ Replication Controller and ReplicaSets have same purpose, but Replication Controller is older way to handle this. Nowadays, ReplicaSets is widely used.

⌘ ReplicaSets is there in **apps/v1** (previously we were using *v1*)

⌘ All the things are same in both *ReplicaSets* and *Replication Controller* yaml configs.

Just, one extra key **selector** (under spec i.e. parent spec, not inside template's spec) will be there.

[it is sibling of template, replicas]

⌘ This is mentioned to tell the ReplicaSets that which Pods come under this.

⌘ But, **why is this required where we are already giving the Pod's template inside it?**

It is because if the Pods are already created before and you want them to come under this.

⌘ **selector** works mainly on labels, means it'll select the pods according to the key and values inside respective *labels* of the pods.

- ⌘ **matchLabels** and **matchExpressions** are there.

```
matchLabels:  
  app: myapp
```

it'll check for the exact match, i.e. Pods having value 'myapp' against the key 'app' inside their *labels* will be selected.

```
matchExpressions:  
  - key: env  
    operator: In  
    values:  
      - prod  
      - staging
```

it is advanced filtering. Pods having value 'prod' or 'staging' against the key 'env' inside their *labels* will be selected.

- ⌘ **NOTE:**

Selectors don't work only for Pods — they work for different resources, but they always select based on labels.

- ⌘ Replication Controller also has this *selectors* options but user input is not mandatory there, if its not mentioned then it'll take all the pods whose labels are matching with the one written in the yaml config of this replication controller's template.

- ⌘ Lets say you gave **replicas** as 4, and there already 4 pods of same type running. then when you give the *selector*, it'll not create any other pod because already 4 are there. So instead of creating new pods, it'll manage those.

And if any pod among them goes down, it'll create one more.

- ⌘ If you have configured some replicas and its running, and want to scale now. Then you'll have the below options

- ⌘ Update the yaml and run *kubectl apply* command.

```
kubectl apply -f <yaml file name>
```

- ⌘ Run the below command without updating the yaml file.

```
kubectl scale --replicas=6 -f <yaml file name>
```

[NOTE: It'll **not update** the yaml file to replica = 6]

- ⌘ Or the below command without mentioning the yaml file.

```
kubectl scale --replicas=6 replicaset <existing replicaset name>
```

(in our case, replicaset name will be *myapp-replicaset*)

```
kubectl scale --replicas=6 replicaset myapp-replicaset
```

TYPE NAME

(that, replicaset in between is the type, what do you want to scale)

```
kubectl create -f replicaset-definition.yml
```

```
kubectl create -f replicaset-definition.yml
```

```
$ kubectl delete replicaset myapp-replicaset
```

*Also deletes all underlying PODs

```
$ kubectl replace -f replicaset-definition.yml
```

```
kubectl scale --replicas=6 -f replicaset-definition.yml
```

NOTE

- The **labels** inside the **selector** and **labels** inside the **template** (i.e. Pod) must be same.
- Otherwise, when you run, even if it creates the pod with the labels defined inside template, it'll not match with the **selector's labels**.
So, it'll think that those pods are not its own, and it'll keep creating the pods.
- However, currently it'll give error saying **selector's labels** and **template's labels** are not matching.
- Template: creation of pods
Selector: owning those pods
- Labels of the replicaset can be anything, that doesn't need to be matched.

```
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      env: production # 1
  template:
    metadata:
      name: nginx-2
      labels:
        env: production # 2
    spec:
      containers:
        - name: nginx
          image: nginx:alpine
```


- See the below

```

apiVersion: v1
kind: Pod
metadata:
  name: nginx-2
  labels:
    env: production
spec:
  containers:
    - name: nginx
      image: nginx:alpine

```

```

apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      env: production # 1
  template:
    metadata:
      name: nginx-2
      labels:
        env: production # 2
    spec:
      containers:
        - name: nginx
          image: nginx:alpine

```

- I had 2 yamls, one for **Pod** and the other for **ReplicaSet**
- I ran one pod using that Pod yaml.

```

$ kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
nginx-2       1/1     Running   0           3s

```

- You can see, Pod's labels and ReplicaSet's selector's labels are matching.
Also, ReplicaSet's selector's labels and template's labels are matching.
- Now, when I run the replicaset using this yaml, it'll create 2 more pods.
Because required is 3, and 1 is already running (nginx Pod script).

```

$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
myapp-replicaset-c2hx7             1/1     Running   0           4s
myapp-replicaset-hl76d             1/1     Running   0           4s
nginx-2                             1/1     Running   0          26s

```

- The pods started from replicaset are having *myapp-replicaset* as the prefix,
because the replicaset's name is *myapp-replicaset*.

- Now, if I delete that nginx-2 pod, then it should create one more.

```
$ kubectl delete pod nginx-2
pod "nginx-2" deleted from default namespace
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-replicaset-c2hx7	1/1	Running	0	6m11s
myapp-replicaset-hl76d	1/1	Running	0	6m11s
myapp-replicaset-sh8cm	1/1	Running	0	5s

- Now, if you create one pod using that Pod yaml file, it'll be created. But it'll be deleted by our replicaset immediately because the required number of pods is 3, not 4.

NOTE

- Lets say one pod is having n number of key value pairs in its labels like

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-2
  labels:
    env: production
    key1: value1
    key2: value2
spec:
```

- And, in the replicaset we are writing 2 key value pairs like

```
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: myapp-replicaset
  labels:
    app: myapp-rs
spec:
  replicas: 3
  selector:
    matchLabels:
      env: production
      key1: value1
  template:
```

- Now, all the Pods having labels both **env: production** and **key1: value1** will be selected.

If any Pod contains only 1 of these or none of these then it'll not be selected.

- Simply, the selector works on 'AND' concept, not 'OR'.

- ♣ If you execute the command

kubectl edit replicaset <replicaset name>

it'll open a yaml file (that is being created in memory for the currently running replicaset) and you can edit there. It'll directly reflect on the running services.

It'll look something like this

```
# Please edit the object below. Lines beginning with a '#' will be ignored,
# and an empty file will abort the edit. If an error occurs while saving this file will be
# reopened with the relevant failures.
#
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"apps/v1","kind":"ReplicaSet","metadata":{"annotations":{},"labels":{"
eplicas":3,"selector":{"matchLabels":{"env":"production"}},"template":{"metadata":{"labels
nx:alpine","name":"nginx"}}}}}}}
  creationTimestamp: "2026-04-24T21:14:15Z"
  generation: 1
  labels:
    app: myapp-rs
  name: myapp-replicaset
  namespace: default
  resourceVersion: "25243"
  uid: 127b82e7-3351-429b-b6ed-4f764dd07062
spec:
  replicas: 3
  selector:
    matchLabels:
      env: production
```

- I updated the *replicas* to 6, and now 6 pods are running

```
$ kubectl get pod
```

NAME	READY	STATUS	RESTARTS	AGE
myapp-replicaset-52k7v	1/1	Running	0	3s
myapp-replicaset-5pmr6	1/1	Running	0	3s
myapp-replicaset-c2hx7	1/1	Running	0	23m
myapp-replicaset-h176d	1/1	Running	0	23m
myapp-replicaset-mhmfv	1/1	Running	0	3s
myapp-replicaset-sh8cm	1/1	Running	0	17m

- NOTE: it'll not change our yaml file. This yaml file is a temporary version that is being there in memory still the replicaset is running (or not deleted). once the replicaset is gone, this temporary yaml will also go.

- | | | |
|---|---|--|
| |  | |
|  | F | |
|  | F | |
|  | F | |
|  | F | |

F

F

F

F

➤ **F**

► **F**

► **F**

► **F**

► **F**

► **F**

► **F**

Debugging Tips

- If the container is not running inside the pods, then see the events at the end after executing the command
kubectrl describe pod <pod name>
-
- F

Commands

➤ **kubectl get nodes -o wide**

```
controlplane ~ ➔ kubectl get nodes -o wide
NAME          STATUS    ROLES    AGE   VERSION   INTERNAL-IP   EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION   CONTAINER-RUNTIME
controlplane   Ready     control-plane   30m   v1.35.0+k3s3   192.168.18.136   <none>        Alpine Linux v3.16   6.8.0-1047-gcp   containerd://1.6.8

controlplane ~ ➔ kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
controlplane   Ready     control-plane   30m   v1.35.0+k3s3
```

it gives extra details about the nodes.

⚡ You can see the OS-IMAGE using

cat /etc/os-release

but, for this you need to have access to that node or you need to do ssh to that node and do this.

⚡ On the other hand, **kubectl** command has all the nodes registered and without ssh or something else it can provide you the details.

➤

➤ F

➤ F

➤ F

➤